Pricing Baku Real Estate from Scratch: Decision Trees and Pegasos SVMs on bina.az Listings

Melek Askerova Sevda Gurbanova Nezrin Ibrahimova

MLE-AI-201 Machine Learning, AI Academy — Cohort II

***Abstract*—**We study two prediction tasks on 58,007 residential sale listings scraped from bina.az: regression of the asking price (Task A) and classification of a derived price tier, premium vs. standard, split at the training median of 208,000 AZN (Task B). We implemented a CART-style decision tree (Gini, entropy and MSE criteria) and a soft-margin SVM trained with the Pegasos sub-gradient method from scratch in NumPy. We also added random Fourier features (RFF) to approximate an RBF kernel. All models share one stratified 70/15/15 split. Hyperparameters are selected on validation only, and every model is compared with its scikit-learn counterpart under matched settings. Our tree reaches test $R^2 = 0.709$ on price (sklearn: 0.710) and F1 $= 0.926$ on the tier task (sklearn: 0.925). Linear Pegasos reaches the same objective value as `LinearSVC` (F1 0.905 vs. 0.906). RFF Pegasos is the best SVM (F1 0.912, AUC 0.974) and matches an exact RBF `SVC`. Trees win on F1 because they capture area$\times$district interactions; the SVMs rank listings better (higher AUC). Errors concentrate on luxury listings, which the tree under-prices by about 38%, and on rare districts. One command reproduces every number: `github.com/nezrinibrahimova/ML_Final_Project` (tag `v1.0-final`).

***Index Terms*—**decision tree, support-vector machine, Pegasos, random Fourier features, regression, classification, real-estate price prediction.

I. Introduction

Asking prices on Baku's largest listing site, bina.az, vary by more than two orders of magnitude, from 16.5k to 5.95M AZN. Size, building type and, above all, location drive them. A model that predicts the price, or says whether a listing belongs to the expensive half of the market, is useful to buyers and sellers. It is also a good test bed for two classic learners whose inductive biases differ: axis-aligned trees and maximum-margin linear classifiers.

Our contributions are: (i) a from-scratch NumPy decision tree for both tasks, with a vectorised split search and four overfitting controls; (ii) a from-scratch Pegasos SVM with a stable learning-rate offset, margin and support-vector reporting, and an RFF kernel approximation; (iii) a single evaluation protocol shared by every model, with matched scikit-learn baselines, 5-fold CV and timing; (iv) an error analysis by price segment, district and market cluster; (v) two bonuses: closed-form Ridge regression, and PCA with k-means market segmentation.

II. Background

A decision tree recursively splits the feature space with axis-aligned tests $x_j \le \theta$. At every node it chooses the test that maximises the impurity decrease $\Delta I = I(P) - \frac{N_L}{N} I(L) - \frac{N_R}{N} I(R)$. Here I is the Gini index $1 - \sum_k p_k^2$, the entropy $-\sum_k p_k \log_2 p_k$ (its decrease is the information gain) or, for regression, the variance (MSE) [1]. Trees need no feature scaling and model interactions naturally, but deep trees have high variance.

The soft-margin SVM finds the hyperplane with the widest margin $2/\|\mathbf{w}\|$ while paying a hinge loss for points inside the margin. Pegasos [2] solves its primal by stochastic sub-gradient descent, which avoids a QP solver. Random Fourier features [3] approximate a shift-invariant kernel with an explicit finite-dimensional map, so a linear SVM can learn an approximately RBF-kernel decision boundary.

III. Data and Preprocessing

Source. The Kaggle *binaaz-sale-project* dump contains 100,775 rows and 51 columns, scraped between 5 Oct and 19 Nov 2024. All prices are in AZN, so no currency conversion is needed.

Cleaning (`src/data_prep.py`). (1) We keep the three residential categories (new build, old build, house/villa; 91,567 rows) and drop land, commercial, office and garage listings. (2) Listings appear repeatedly across scrapes, so we keep the latest record per listing URL (58,452 rows). (3) We parse area, floor/total floors and land size from their text fields. (4) Outlier rules: area 15–1000 m^2, price per m^2 300–10,000 AZN (computed only for filtering and then discarded), land $\le$ 1000 sot, valid room count, and coordinates inside Azerbaijan. (5) Missing floor and land values become 0 (houses have no floor; flats have no land). This leaves 58,007 listings.

Leakage. `unit_price` (price per m^2) and `total_price` are functions of the target and are dropped. Free text, owner and shop names, raw address, view counts and image URLs are not used as features either.

Features. Ten numeric features (area, rooms, floor, total floors, land, latitude, longitude, and repair/deed/mortgage flags) plus one-hot category and district. The one-hot columns are defined on the training split only (unseen levels map to all-zeros), giving 148 features.

Exploratory analysis (`src/eda.py`). Prices are heavily right-skewed (median 208,000, mean 263,374, max 5.95M AZN; skewness 5.79). Taking the logarithm makes the distribution almost symmetric (skewness 0.27, Fig. 1). Area is the strongest single driver (correlation 0.73 with price), but the building type changes the price level: new builds have a median of 240k AZN at 101 m^2, old builds 145k at 65 m^2, and houses/villas 200k at 180 m^2. Location matters



Figure 1. Distribution of $\log(1+\text{price})$ after cleaning. The raw price is strongly right-skewed (skewness 5.79); the log is close to symmetric (0.27).

Table 1. Dataset summary after cleaning.

Property	Value
Raw rows / columns	100,775 / 51
Rows after cleaning	58,007
Features used	148 (10 numeric + one-hot)
Districts	138
Train / Val / Test	40,605 / 8,701 / 8,701
Tier threshold (train median)	208,000 AZN
Premium share (train/val/test)	50.0 / 50.0 / 50.0 %

just as much. Median prices range from about 30k in remote settlements to 510k in Ağ şəhər and 390k in Mərdəkan, and 56 of the 138 districts have fewer than 50 listings. Two- and three-room flats make up 71% of the market.

Targets and split. Because of this skew the regression tree is trained on $\log(\text{price})$, and its metrics are reported in AZN. The tier label is premium if the price exceeds the *training* median (208,000 AZN). One stratified split on price quartiles (seed 42) is shared by every model (Table 1). A unit test asserts that the tree and SVM scripts use identical rows.

IV. Method

A. Decision tree (from scratch)

`DecisionTree` (`src/decision_tree.py`) supports classification with Gini or entropy and regression with MSE. Growth is recursive. A node becomes a leaf when it is pure, when `max_depth` is reached, when it has fewer than `min_samples_split` samples, or when the best split's weighted impurity decrease $\frac{N}{N_{\text{tot}}}\Delta I$ is below `min_impurity_decrease`. `min_samples_leaf` masks candidate splits that would create a smaller child. A unit test checks that each of the four hyperparameters changes the fitted tree.

Split search. For every feature we sort once and compute cumulative class counts (classification) or cumulative $\sum y$ and $\sum y^2$ (regression). Every candidate threshold is then scored in $O(1)$ and all thresholds at once, in vectorised form, so a node costs $O(d\,n\log n)$. Thresholds are midpoints between distinct consecutive values, as in scikit-learn. A guard handles adjacent floats whose midpoint rounds up to the larger value, which would otherwise create an empty child; a regression test covers this case.

Outputs. Leaves store the class frequencies, which give `predict_proba`, or the mean target. `feature_importances_` sums each feature's weighted impurity decreases and normalises them. A unit test checks that our root split, predictions and importances match scikit-learn on small deterministic data.

B. Support-vector machine (from scratch)

Our `PegasosSVM` (`src/svm.py`, NumPy only) minimises the soft-margin primal with labels $y_i \in \{-1,+1\}$ (premium $=+1$):

$$J(\mathbf{w},b) = \tfrac{\lambda}{2}\|\mathbf{w}\|^2 + \tfrac{1}{n}\sum_{i=1}^{n}\max\bigl(0,\,1-y_i(\mathbf{w}^\top\mathbf{x}_i+b)\bigr). \quad (1)$$

At step t we draw a mini-batch A_t of size k and let $A_t^+ = \{i \in A_t : y_i(\mathbf{w}^\top\mathbf{x}_i+b) < 1\}$ be the points with active hinge. The sub-gradient update is

$$\mathbf{w} \leftarrow (1-\eta_t\lambda)\,\mathbf{w} + \tfrac{\eta_t}{k}\textstyle\sum_{i\in A_t^+} y_i\mathbf{x}_i, \quad (2)$$

$$b \leftarrow b + \tfrac{\eta_t}{k}\textstyle\sum_{i\in A_t^+} y_i. \quad (3)$$

The bias is not regularised, so it receives a plain sub-gradient step. Each epoch is one shuffled pass over the training set; every few steps we record $J(\mathbf{w},b)$ on the full training set to show that it decreases.

Learning rate. The paper's schedule $\eta_t = 1/(\lambda t)$ [2] makes the first step $\eta_1 = 1/\lambda$ (e.g. 10^3 for $\lambda = 10^{-3}$). The factor $(1-\eta_t\lambda)$ pulls $\mathbf{w}$ back after such a step, but nothing pulls the unregularised bias back. On separable toy data one early violator moved b to ≈ 370, and it was still there after 10 epochs. We therefore use $\eta_t = 1/(\lambda(t+t_0))$ with $t_0 = 1/\lambda$, so that $\eta_1 \approx 1$. We also implemented $t_0 = 0$, $\eta_0/\sqrt{t}$ and a constant rate, and compare them in Fig. 4.

Margin and support vectors. After training we report $\|\mathbf{w}\|$, the geometric margin $2/\|\mathbf{w}\|$ and the support vectors, i.e. the training points with $y_i(\mathbf{w}^\top\mathbf{x}_i+b) \le 1$ (on or inside the margin, or misclassified).

Non-linear extension: random Fourier features. Following Rahimi and Recht [3], we map $\phi(\mathbf{x}) = \sqrt{2/D}\cos(W^\top\mathbf{x}+\mathbf{c})$ with $W_{jk} \sim \mathcal{N}(0,2\gamma)$ and $c_k \sim U[0,2\pi]$, so that $\phi(\mathbf{x})^\top\phi(\mathbf{x}') \approx \exp(-\gamma\|\mathbf{x}-\mathbf{x}'\|^2)$. Linear Pegasos on $\phi(X)$ then approximates an RBF-kernel SVM; a unit test checks the kernel approximation error (< 0.05 at $D = 20{,}000$). We chose RFF over kernelised Pegasos because one epoch costs $O(nD)$ instead of $O(n^2)$ kernel evaluations. With $n \approx 4\times10^4$ training rows this difference matters, and prediction does not need the support vectors.

Feature scaling. Pegasos is not scale-invariant, so features are standardised with training statistics only. For the linear SVM we standardise every column. For the RBF models (RFF and the sklearn `SVC` baseline) we standardise only the numeric columns and keep the one-hot district/category columns as 0/1. A standardised rare district dummy becomes ≈ 200 for the few listings in that district, and squared distances are then dominated by district identity. Section B measures this effect.

Table 2. Headline test results (8,701 listings). Task A in AZN; Task B for the premium class. CV: 5-fold on the training split.

Task A: price regression Model	RMSE	MAE	R^2
Train-mean baseline	222,308	131,332	0.000
Ours: Ridge (bonus)	124,377	57,792	0.687
Ours: decision tree (log target)	119,931	43,889	0.709
sklearn tree	119,699	43,852	0.710
Task B: price tier Model	F1	AUC	CV F1
Ours: tree, Gini	.923	.954	$.920 \pm .004$
Ours: tree, entropy	**.926**	.930	$.920 \pm .003$
sklearn tree, Gini	.921	.953	$.920 \pm .004$
sklearn tree, entropy	.925	.929	$.920 \pm .004$
Ours: Pegasos linear	.905	.964	$.904 \pm .005$
Ours: Pegasos + RFF	.912	**.974**	n/a
sklearn `LinearSVC`	.906	.964	$.905 \pm .005$
sklearn `SGDClassifier`	.906	.964	$.905 \pm .005$
sklearn `SVC` RBF (10k)	.912	.973	n/a

V. Experimental Setup

Metrics are implemented in NumPy (`src/evaluate.py`): RMSE, MAE and R^2 in AZN for Task A (plus RMSE and R^2 on the log scale), and precision, recall and F1 for the premium class, plus rank-based (Mann–Whitney, tie-aware) ROC-AUC, for Task B. AUC uses leaf frequencies for trees and signed margins for SVMs. Unit tests check every metric against hand-computed values, including edge cases such as a constant target for R^2. The data pipeline has its own tests: cleaned row count, coordinate bounds, split reproducibility, and train-only standardisation. In total, 49 tests cover data, metrics, tree, SVM and bonuses.

Protocol. Every hyperparameter is chosen on the validation split and the test split is scored once, for the final configurations only. Trees: depth and leaf sweeps for each criterion (Fig. 3), then a joint search over `max_depth` $\in \{10,12,15,20\} \times$ `min_samples_leaf` $\in \{1,3,5,10\}$. SVMs: a λ sweep, then a $\gamma \times \lambda$ sweep for RFF and a sweep over the number of random features D.

Baselines. scikit-learn `DecisionTreeRegressor/Classifier` use the same selected depth, leaf size and criterion. `LinearSVC` (hinge) and `SGDClassifier` (hinge, $\alpha = \lambda$) use $C = 1/(\lambda n)$ so that the objective equals Eq. (1). `SVC` (RBF) uses the selected γ and is fitted on a stratified 10k-row subsample, because exact kernel training is $O(n^2)$. Each model is also evaluated with 5-fold CV on the training split; scaling is fitted inside every fold (an sklearn `Pipeline` for the baselines). We report fit and predict times on one laptop CPU.

Reproducibility. Every source of randomness uses seed 42. `python -m src.run_all` rebuilds every number, table and figure from a fresh clone with the pinned `requirements.txt` in about 24 minutes; two independent runs produced identical tables.

VI. Results

Table 2 is the headline comparison on the test split.

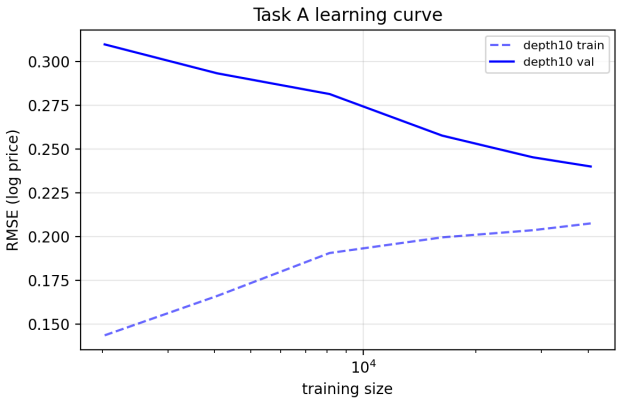


Figure 2. Task A learning curve of our regression tree (depth 10): train (dashed) and validation (solid) RMSE of log(price) against the number of training listings.

A. Decision tree

Hyperparameters. Fig. 3 shows the classic bias–variance picture. Training F1 climbs to 0.99 at depth 20, while validation F1 rises steeply up to depth 6 and then flattens (0.920–0.923 at depth 12, 0.925–0.927 at 20). Gini and entropy are within 0.007 of each other at every depth. The joint search selected depth 20 for all three trees, with leaf sizes 10 (regression), 5 (Gini) and 1 (entropy); validation scores were $\text{RMSE}_{\log} = 0.223$, F1 0.927 and 0.927. Depth 20 is the edge of our grid, but the validation gain from 15 to 20 is only about 0.002 F1, so a deeper grid would change little.

Comparison with scikit-learn. With identical settings our trees reproduce scikit-learn closely. Leaf counts are 2,841 vs. 2,837 (regression), 1,506 vs. 1,511 (Gini) and 2,326 vs. 2,350 (entropy). Test predictions agree on 99.5% (Gini) and 98.3% (entropy) of listings, and the regression predictions correlate at 0.9995. 9–10 of the top-10 important features coincide; area dominates (importance 0.62), followed by longitude and latitude. The small differences come from ties: when two splits have exactly the same impurity decrease, scikit-learn breaks the tie with a random feature permutation, while we keep the first. CV confirms the match (Table 2). Our tree is 7–22$\times$ slower to fit (15.3 s vs. 2.1 s on the full training set), the price of Python-level recursion compared with Cython.

Regression tree and learning curve. On Task A the tree explains 86.5% of the variance of log(price) (test $R^2_{\log} = 0.865$) and 70.9% in AZN. The lower AZN value is driven by the few very expensive listings, whose absolute errors dominate the squared loss. The learning curve (Fig. 2) shows the validation error still falling at the full training size while the training error rises toward it: a variance-dominated model that would benefit from more data. The most important features are area (0.62), longitude (0.12) and latitude (0.11): location enters mainly through coordinates rather than through individual district dummies.

Gini vs. entropy. The entropy tree has the best F1 (0.926) but a much lower AUC (0.930 vs. 0.954). With `min_samples_leaf`$= 1$ its leaves are pure, so almost every probability is 0 or 1, which gives a poor ranking. The Gini tree (leaf size 5) is the better-calibrated choice at the same F1.

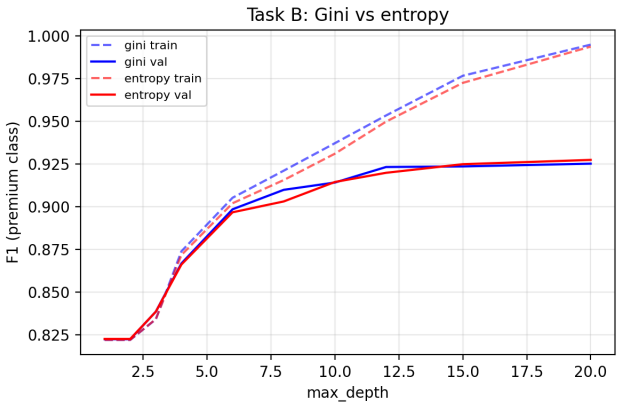


Figure 3. Task B tree: train (dashed) and validation (solid) F1 against `max_depth` for Gini and entropy.

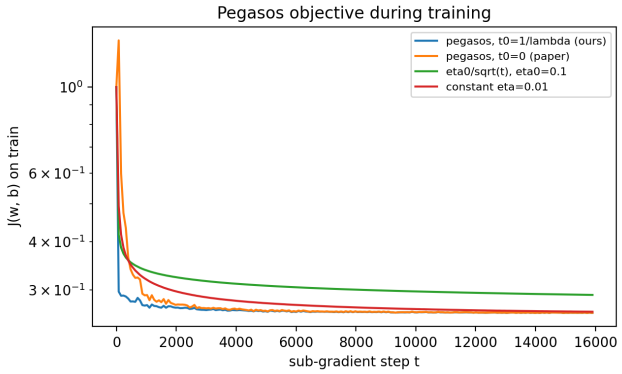


Figure 4. Training objective $J(\mathbf{w}, b)$ of linear Pegasos ($\lambda = 10^{-3}$) under four learning-rate schedules.

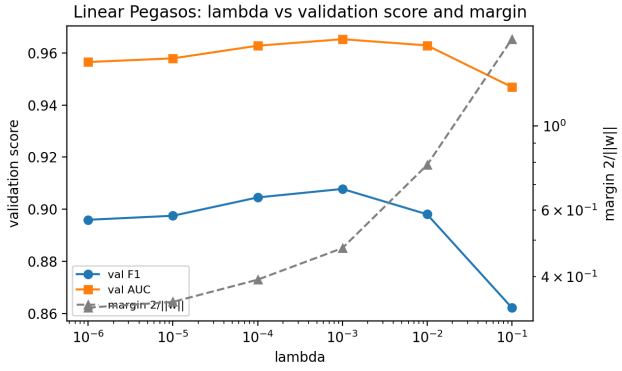


Figure 5. Linear Pegasos: validation F1/AUC (left axis) and margin $2/\|\mathbf{w}\|$ (right axis, log) against λ.

B. SVM (Task B)

Protocol. We sweep $\lambda \in \{10^{-6}, \ldots, 10^{-1}\}$ for linear Pegasos (100 epochs, $k = 256$). For RFF Pegasos (30 epochs, $k = 64$) we sweep $\gamma \in \{0.01, 0.03, 0.1, 0.3\} \times \lambda \in \{10^{-6}, 10^{-5}, 10^{-4}\}$ at $D = 1000$, then vary D; the final model uses $D = 2000$. All choices maximise validation F1.

The from-scratch solver reaches the optimum. At the selected $\lambda = 10^{-3}$, Pegasos ends at $J = 0.2619$, which equals the objective of the `LinearSVC` solution. Test F1 differs by 0.001, and the CV F1 difference is 0.0008, well inside one standard deviation (Table 2). Fig. 4 shows J decreasing. With the paper's schedule ($t_0 = 0$), J first jumps above its starting value of 1 before recovering, while our offset schedule drops at once. $\eta_0/\sqrt{t}$ is too slow at the same budget ($J = 0.291$, val F1 .889).

Margin trade-off. Increasing λ widens the margin $2/\|\mathbf{w}\|$ from 0.33 ($\lambda = 10^{-6}$) to 1.69 ($\lambda = 10^{-1}$). The number of support vectors grows from 9.1k to 19.8k (22% $\rightarrow$ 49% of the training set). Validation F1 peaks at $\lambda = 10^{-3}$ (margin 0.48, F1 .908) and falls to .862 at the widest margin, where the model underfits (Fig. 5).

Non-linearity helps. RFF Pegasos is the best SVM on test (F1 .912, AUC .974). It matches the exact RBF `SVC` and keeps training linear in n. Validation F1 rises with the number of random features: .888 at $D = 250$, .901 at 500, .913 at 1000 and .917 at 2000. The approximation is the bottleneck at small D. The selected bandwidth is $\gamma = 0.1$; a larger $\gamma = 0.3$ overfits to local neighbourhoods (val F1 .890). Feature scaling matters as much as the kernel. With every column standardised (largest $|x| = 202$, a rare district dummy), the same RFF model reaches only val F1 .859 / AUC .933. With one-hot columns kept as 0/1 it reaches .913 / .973.

What the linear SVM uses. The weights of the standardised features are dominated by `area_m2` ($w = 3.7$). The next largest weights are district effects: outer settlements (Masazır, Binəqədi, Binə) push toward *standard*, central districts (Nizami, 28 May, Ağ şəhər) toward *premium*, and new-build vs. house/villa category matters as well.

C. Error analysis

Luxury listings. The regression tree's median absolute percentage error is 8.0% for the cheaper half of the test set and 8.3% for the 50–90th percentiles. It rises to 15.2% for the 90–99th percentiles and 34.7% for the top 1% (mean price 1.7M AZN). There the mean log error is -0.48, so the tree under-prices luxury listings by about 38% (Fig. 6, left). A leaf averages many listings, so the few extreme prices are pulled toward their neighbours.

Rare districts. The districts with the largest errors have few training listings. For example, Xaçmaz (21 training listings) has a median error of 122%, and Qala and Kürdəxanı (20–34 listings) have about 60%. In such districts the tree must fall back on area and coordinates learned elsewhere.

The tier border. Task B errors are mostly near the threshold. The error rate is 27.1% for listings within 10% of 208,000 AZN, 12.8% at 10–25%, and below 4% beyond 25% (Fig. 6, right). Most of the remaining classification error is therefore the irreducible ambiguity of an arbitrary cut on a noisy price, not a modelling failure.

D. Bonus: Ridge regression and market segments

Ridge (P8). Closed-form Ridge, $\hat{\mathbf{w}} = (X^\top X + \alpha I)^{-1} X^\top \mathbf{y}$ on standardised features ($\alpha = 0.01$ on validation), reaches test RMSE 124,377 and $R^2 = 0.687$. It matches sklearn `Ridge` to 2×10^{-5} AZN, but it is clearly behind the tree in MAE (57.8k vs. 43.9k). A linear price model cannot express that one m^2 is worth more in the centre.

Segments (P10). PCA keeps 125 components (90.4% of the variance), and k-means ($k = 5$) on them yields market seg-

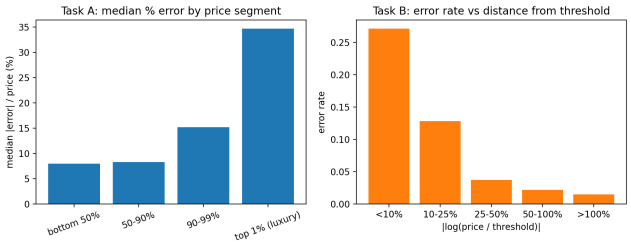


Figure 6. Tree error analysis on test. Left: Task A median percentage error by true-price segment. Right: Task B error rate against distance from the tier threshold.

ments. The hardest segment is the large-house cluster (6,404 training listings, median 180 m^2): validation R^2 is 0.56 for both Ridge and the tree, against 0.65–0.93 in the other clusters. The tree has the lower MAE in four of five clusters. For the tier task the linear SVM has the higher AUC in every cluster, consistent with the ranking advantage in Table 2. These cluster numbers are diagnostics on the validation split, not independent test estimates.

VII. Discussion and Limitations

Trees vs. SVMs. The tree wins on F1 (0.923–0.926 vs. 0.905–0.912). The price tier is largely an interaction of size and location, which axis-aligned splits express directly: within one district, area decides. The SVMs win on AUC (0.964–0.974 vs. 0.930–0.954), because a continuous margin ranks listings smoothly while a tree has a few thousand discrete leaf scores. A user who needs a ranking or a calibrated score should prefer the RFF SVM; a user who needs the hard premium/standard label should prefer the tree.

Bias and variance. The deep trees (depth 20) have training F1 near 0.99 but validation F1 near 0.925, so they have high variance. The Task A learning curve is still improving at 40k listings (validation $\mathrm{RMSE}_{\log}$ falls from 0.245 to 0.240 between 28k and 40k), while Task B has flattened, so averaging trees (a forest) is the more promising remedy. Linear Pegasos is the opposite case: its CV, validation and test F1 all lie within 0.005 (0.904–0.908), so its errors are mostly bias. RFF trades some of that bias for variance and gains 0.007 F1.

Limitations. (i) Asking prices are not transaction prices. (ii) The data covers six weeks of 2024, so we cannot measure temporal drift. (iii) Districts with fewer than $\sim$50 listings are poorly served by every model. (iv) The exact RBF `SVC` baseline saw only 10k training rows, so the comparison with RFF (40k rows) favours ours. (v) Our tree is about 7$\times$ slower than scikit-learn. (vi) The tree grid stops at depth 20.

VIII. Conclusion

From-scratch NumPy implementations of a CART tree and a Pegasos SVM match scikit-learn under one leakage-free protocol: tree R^2 0.709 vs. 0.710, and the same SVM objective value. On bina.az listings, trees are the most accurate classifiers of the price tier (F1 0.926), while an RFF-approximated RBF SVM gives the best ranking (AUC 0.974). Three choices mattered as much as the model: a stable Pegasos step size, keeping one-hot features unscaled for RBF kernels, and a log target for prices. Next we would add a random forest of our own trees to cut the tree's variance, add district-level target statistics learned on training data only for rare districts, and model luxury listings separately.

Team Contributions

Nezrin Ibrahimova (A): data loading and cleaning, leakage audit, features, the shared split and tier label (`data_prep.py`), EDA (`eda.py`), all metrics (`evaluate.py`), and the bonuses: Ridge regression, PCA/k-means segmentation and the matched cluster error analysis. *Sevda Gurbanova (B)*: the from-scratch decision tree (`decision_tree.py`), its tests, the hyperparameter sweeps and learning curves, the comparison with scikit-learn, CV, and the tree error analysis. *Melek Askerova (C)*: the from-scratch Pegasos SVM with random Fourier features (`svm.py`), its tests and experiments, the SVM baselines, the single `run_all` entry point, pinned dependencies and packaging.

Tools and Acknowledgements

Data: the Kaggle dataset *binaaz-sale-project* by `sehriyarmemmedli`. Libraries: NumPy, pandas, Matplotlib, and scikit-learn, used only for baselines, CV splitting and the bonus comparisons. AI assistance: an AI coding assistant was used to help write and debug the SVM code and the `run_all` integration, and to draft parts of this report. Every number was produced by `run_all` and checked by the team.

References

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone, *Classification and Regression Trees*. Wadsworth, 1984.

[2] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter, "Pegasos: primal estimated sub-gradient solver for SVM," *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011.

[3] A. Rahimi and B. Recht, "Random features for large-scale kernel machines," in *Advances in Neural Information Processing Systems*, vol. 20, 2007.